

Arquitectura del Computador

Práctica de Laboratorio 4

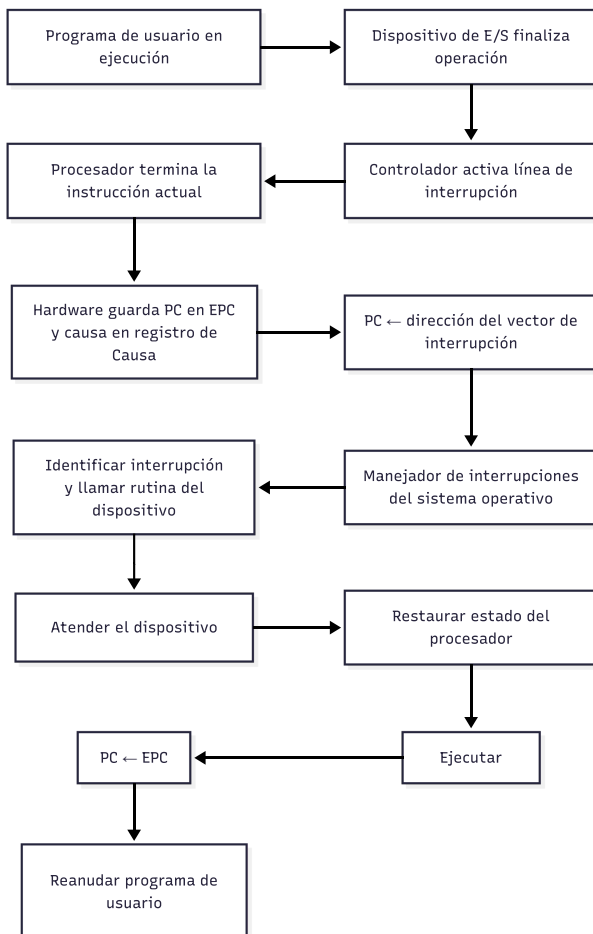
Angel Navas y Jose Natera
Departamento de Computación
Universidad de Carabobo
Barbula, Carabobo

Abstract—Familiarizar al estudiante con la sintaxis del lenguaje ensamblador MIPS32 y su ejecución en el entorno MARS, resolviendo problemas clásicos de programación, prestando especial al uso de interrupciones para la gestión de entrada/salida (E/S) y la utilización de rutinas de servicio para responder a eventos como entrada por teclado y control de tiempo, simulando el funcionamiento de periféricos.

Index Terms—Computer Architecture, Assembly Language, I/O Operations, Time Management, Hardware Simulation

I. INFORME

A. Explique con un diagrama cómo funciona el ciclo de una interrupción de hardware (desde que ocurre el evento hasta que se atiende).



B. ¿Qué diferencias hay entre gestionar E/S con sondeo y hacerlo con interrupciones?

La principal diferencia entre la gestión de entradas y salidas por sondeo (polling) y por interrupciones en el entorno MIPS32 radica en la eficiencia del uso del procesador y la carga sobre el flujo de ejecución. En el método de sondeo, el procesador ejecuta un bucle continuo consultando repetidamente el registro de estado del periférico para verificar si una operación se ha completado, lo que desperdicia ciclos de reloj en una espera activa que impide realizar otras tareas. Por el contrario, el manejo por interrupciones permite que el procesador ejecute otras instrucciones del programa principal de manera ininterrumpida hasta que el hardware genera una señal externa; en ese momento, el procesador suspende su flujo actual, guarda el estado del programa y salta a una rutina de servicio de interrupción (ISR) para atender el evento, optimizando así el rendimiento global del sistema al eliminar la espera innecesaria. Al implementar estos conceptos en el simulador MARS, se observa que el sondeo requiere una lógica de control simple basada en instrucciones de carga y saltos condicionales que monitorean constantemente una dirección de memoria específica. Sin embargo, la gestión por interrupciones introduce una complejidad técnica mayor, ya que exige configurar el registro de estado del coprocesador 0, habilitar los bits de máscara de interrupción y manejar adecuadamente el registro EPC para retomar la ejecución tras la atención del periférico. Mientras que el sondeo es predecible y fácil de depurar para tareas críticas de tiempo real muy simples, las interrupciones ofrecen una arquitectura mucho más escalable y eficiente para sistemas que deben atender múltiples dispositivos de E/S de forma asíncrona sin degradar la capacidad de procesamiento del núcleo MIPS.

C. ¿Qué ventajas tiene el uso de interrupciones en términos de uso del procesador?

La ventaja fundamental del uso de interrupciones reside en la liberación del procesador de la tarea de vigilancia constante, transformando un modelo de espera activa en uno de notificación asíncrona. En lugar de dedicar ciclos de instrucción a ejecutar bucles que consultan el bit de listo de un periférico, lo cual consume el 100% del tiempo de CPU sin producir trabajo útil, las interrupciones permiten que el procesador se dedique exclusivamente a procesar algoritmos o tareas de fondo. Esta eficiencia se traduce en una capacidad de respuesta superior, ya que el hardware solo reclama la atención del núcleo MIPS

cuando existe un dato real que procesar o una transferencia que finalizar, maximizando así el throughput o tasa de transferencia efectiva del sistema. Además, este enfoque permite una gestión más sofisticada del consumo energético y de la multitarea, ya que el procesador puede incluso entrar en estados de bajo consumo mientras espera una señal externa. Al delegar la detección del evento al hardware, se elimina la latencia intrínseca del sondeo, donde el tiempo de respuesta depende de la frecuencia con la que el software consulta el periférico. En MIPS32, esto permite que la CPU mantenga una alta utilización en procesos computacionales complejos, garantizando que la entrada/salida no se convierta en un cuello de botella que detenga innecesariamente la ejecución de las instrucciones de usuario.

D. ¿Qué registros especiales se utilizan en MIPS32 para gestionar interrupciones?

La gestión de interrupciones en la arquitectura MIPS32 depende fundamentalmente del Coprocesador 0 (CP0), el cual integra registros específicos para el control de excepciones y estados del sistema. Entre los más críticos se encuentra el registro \$12 (Status), que actúa como el panel de control principal para habilitar o deshabilitar interrupciones tanto a nivel global como para dispositivos específicos mediante máscaras de bits. Por otro lado, el registro \$13 (Cause) es esencial para identificar el origen de la interrupción, ya que contiene un campo de código de excepción que indica si el evento fue provocado por un fallo de hardware, una instrucción no válida o una solicitud de un periférico externo en el simulador MARS. Complementando esta estructura, el registro \$14 (EPC - Exception Program Counter) cumple el rol vital de almacenar la dirección de la instrucción que se estaba ejecutando en el momento exacto en que ocurrió la interrupción, permitiendo que el procesador retome su tarea original una vez finalizada la rutina de servicio. Asimismo, el registro \$8 (BadVAddr) se utiliza frecuentemente en conjunto con estos para capturar direcciones de memoria erróneas en caso de que la interrupción sea producto de un fallo de direccionamiento. En la práctica dentro de MARS, estos registros no se manipulan con instrucciones de carga estándar como lw, sino mediante instrucciones privilegiadas como mfc0 (move from coprocessor 0) y mtc0 (move to coprocessor 0), asegurando un aislamiento entre el código de usuario y el manejo crítico del sistema.

E. ¿Por qué es necesario guardar el contexto (registros) al entrar en una rutina de servicio?

La necesidad de guardar el contexto radica en que la rutina de servicio de interrupción (ISR) utiliza los mismos registros de propósito general que el programa principal que estaba en ejecución. Dado que una interrupción es un evento asíncrono e impredecible, el programa interrumpido no tiene forma de saber que sus datos están a punto de ser sobrescritos por la lógica de la ISR. Si no se preservara este estado en la pila, al finalizar la interrupción y ejecutar la instrucción eret, el procesador regresaría a un flujo de trabajo donde los valores de las variables temporales y de control habrían sido alterados arbitrariamente, provocando fallos de segmentación, cálculos

erróneos o el colapso total del software. En la arquitectura MIPS32, este proceso de salvaguarda garantiza la transparencia de la ejecución, permitiendo que el hardware simule que el programa principal nunca se detuvo. El programador debe diseñar la rutina para que lo primero que haga sea desplazar el puntero de pila \$sp y almacenar todos los registros que planea utilizar, incluyendo el registro de dirección de retorno \$ra si la ISR realiza llamadas a otras funciones. Al restaurar estos valores justo antes de salir de la excepción, se asegura que el hilo de ejecución original encuentre el procesador en el estado exacto en que lo dejó, manteniendo la integridad de los datos y la continuidad de la lógica de los ejercicios que estamos desarrollando en MARS.

F. Momentos en que pueden generarse excepciones en un sistema MIPS32. a) Enumera al menos 4 situaciones en las que se pueda generar una excepción (por ejemplo: desbordamiento aritmético, fallo de dirección, etc.). b) Explica qué etapas del pipeline pueden provocar una excepción y por qué.

En el ecosistema MIPS32, las excepciones pueden ser disparadas por una variedad de eventos internos y externos que interrumpen el flujo secuencial de instrucciones para garantizar la integridad del sistema. Entre las situaciones más comunes se encuentra el desbordamiento aritmético (overflow), que ocurre cuando el resultado de una operación con signo excede la capacidad del registro, y el fallo de dirección (address error), generado al intentar acceder a una dirección de memoria no alineada o protegida. Asimismo, se presentan excepciones por instrucciones ilegales o reservadas, donde la Unidad de Control no reconoce el código de operación (opcode), y las llamadas al sistema (syscalls), que son excepciones voluntarias utilizadas para solicitar servicios al kernel o al simulador MARS, como la impresión de datos en consola. Dentro del pipeline de cinco etapas de MIPS32, las excepciones pueden originarse en casi cualquier fase dependiendo de la naturaleza de la instrucción. En la etapa de Búsqueda (IF), se generan excepciones si la dirección del PC es inválida, mientras que en la de Decodificación (ID) se detectan códigos de operación desconocidos. La etapa de Ejecución (EX) es la responsable de capturar errores aritméticos tras el paso por la ALU, y la etapa de Memoria (MEM) gestiona fallos de alineación o violación de privilegios al intentar leer o escribir en la RAM. Es crucial que el hardware identifique en qué etapa ocurrió el evento para que el registro EPC guarde la dirección exacta de la instrucción conflictiva, permitiendo que el sistema maneje el error de forma precisa antes de que los datos corruptos lleguen a la etapa de Escritura (WB).

G. Estrategias de tratamiento de excepciones e interrupciones a) Explica las diferencias entre interrupciones y excepciones (¿son síncronas o asíncronas?). b) Describe brevemente dos estrategias para tratar excepciones en un sistema MIPS32 ¿Cómo se redirige la ejecución hacia la rutina de servicio? ¿Cuál es la función del registro EPC (Exception Program Counter)?

La distinción fundamental entre interrupciones y excepciones en MIPS32 radica en su origen y su relación con el flujo de instrucciones: las excepciones son síncronas, ya que son

provocadas internamente por la ejecución de una instrucción específica, como una división por cero o un acceso a memoria inválido, ocurriendo siempre en el mismo punto del programa si se repiten las mismas condiciones. Por el contrario, las interrupciones son asíncronas, originándose en dispositivos externos de hardware como un teclado o un temporizador en MARS de forma totalmente independiente al contador de programa, lo que significa que pueden ocurrir en cualquier momento entre la ejecución de dos instrucciones cualesquiera. Para tratar estos eventos, el sistema MIPS32 emplea principalmente dos estrategias: el uso de un manejador único de excepciones con consulta del registro Cause, donde el hardware salta a una dirección fija generalmente 0x80000180 y el software debe preguntar al registro \$13 qué ocurrió, o el uso de vectores de interrupción, donde cada tipo de evento tiene una dirección de salto predefinida en memoria para agilizar la respuesta. En ambos casos, el hardware redirige la ejecución cargando la dirección de la rutina de servicio en el PC y salvando simultáneamente la dirección de la instrucción interrumpida en el registro EPC. La función crítica del EPC es actuar como un marcador de posición, permitiendo que tras finalizar la rutina de tratamiento, la instrucción `eret` recupere ese valor y el procesador retome el programa exactamente donde se detuvo sin pérdida de continuidad.

H. Habilitación de interrupciones en dispositivos y procesador a) Explica cómo se habilitan las interrupciones en: El teclado. La pantalla. El procesador (detalla qué bits del registro Status deben modificarse). b) ¿Qué pasaría si habilitamos interrupciones en los dispositivos, pero no en el procesador?

Para habilitar las interrupciones en los periféricos del simulador MARS, es necesario interactuar directamente con sus registros de control mapeados en memoria. En el caso del teclado, se debe poner a 1 el bit de habilitación de interrupciones (bit 1) en la dirección 0xffff0000 (Keyboard Control Register); de igual forma, para la pantalla, se activa el bit 1 en la dirección 0xffff0008 (Display Control Register). Sin embargo, estas señales no llegarán al núcleo si el procesador no está configurado: en el registro \$12 (Status) del Coprocesador 0, se deben modificar obligatoriamente el bit 0 (Interrupt Enable - IE) para permitir interrupciones globales y los bits del 8 al 15 (Interrupt Mask - IM) para autorizar las líneas específicas de hardware siendo el bit 8 para software y del 10 al 15 para hardware externo que corresponden a los dispositivos conectados. Si se decide habilitar las interrupciones únicamente en los dispositivos, pero se mantuviera el procesador ciego a ellas, el sistema simplemente ignoraría cualquier evento externo. Los periféricos cambiarían sus bits de estado y enviarían la señal eléctrica de interrupción hacia la CPU, pero debido a que el bit IE del registro Status estaría en 0, el procesador no detendría su flujo de ejecución ni saltaría a la rutina de servicio. En la práctica, esto se traduce en un comportamiento idéntico al del sondeo, donde los datos se acumulan en los buffers de los dispositivos, pero el programador tendría que consultarlos manualmente mediante instrucciones `lw`, ya que el mecanismo de notificación auto-

mática está bloqueado por la máscara de control del núcleo MIPS.

I. Procesamiento de interrupciones a) Describe paso a paso qué ocurre cuando se produce una interrupción de reloj: Desde que el evento ocurre hasta que la rutina de servicio termina. ¿Qué registros se guardan y restauran? ¿Qué hace el hardware y qué hace el software (sistema operativo o rutina)? b) ¿Por qué es importante guardar el contexto (registros generales, EPC, Status) al entrar en la rutina?

Cuando se produce una interrupción de reloj en MIPS32, el hardware detecta la señal eléctrica y completa la instrucción en curso antes de iniciar la transición. El hardware congela el flujo, guarda automáticamente la dirección de la siguiente instrucción en el registro EPC, deshabilita las interrupciones en el registro Status (poniendo el bit IE en 0) y carga en el PC la dirección del manejador de excepciones habitualmente 0x80000180. Una vez allí, el software toma el control: su primera tarea es salvar el contexto en la pila, incluyendo los registros de propósito general que se planean usar, el registro Status original y el EPC. Tras procesar la lógica del reloj como actualizar un contador o cambiar de proceso, el software restaura todos los registros desde la pila, recarga el estado original en el Coprocesador 0 y ejecuta la instrucción `eret`, la cual devuelve el control al programa principal usando el valor almacenado en el EPC. La importancia de guardar este contexto radica en que la interrupción es un evento transparente para el programa que estaba corriendo. Si la rutina de servicio modificara el registro \$t0 o \$s0 sin haber guardado su valor previo en la memoria, al regresar al programa principal este se encontraría con datos corruptos, provocando comportamientos erráticos o el cierre del sistema. Guardar el Status es vital para recuperar las máscaras de interrupción y el estado de privilegios previo, mientras que preservar el EPC asegura que, incluso si ocurre otra excepción dentro de la rutina, el sistema no pierda la dirección de retorno original. En esencia, este respaldo de información es el seguro de vida que permite que la multitarea y la gestión de periféricos coexistan sin que las tareas de usuario perciban que fueron pausadas.

J. Interrupciones de reloj y control de ejecución a) Explica cómo una interrupción de reloj puede usarse para: Evitar que un programa quede en un bucle infinito. Finalizar programas que superan un tiempo máximo de ejecución. b) ¿Qué debe hacer el software si el programa finaliza antes de que ocurra la interrupción de reloj?

La interrupción de reloj actúa como un mecanismo de protección de recursos y control de CPU. Para evitar que un programa quede atrapado en un bucle infinito, el hardware del temporizador se configura para generar una interrupción tras un número determinado de ciclos; cuando el tiempo expira, el control salta forzosamente del código de usuario a la rutina de servicio del sistema ISR. En ese punto, el sistema operativo puede inspeccionar el estado del proceso y, si detecta que ha superado su tiempo máximo de ejecución (quantum), puede decidir no restaurar el contexto de ese programa y, en su lugar, liberar la memoria ocupada o saltar a una rutina de finalización, impidiendo que un proceso defectuoso monopo-

lice el procesador indefinidamente. Si un programa finaliza su ejecución de manera exitosa antes de que ocurra la interrupción de reloj programada, el software debe realizar una cancelación o limpieza del temporizador. Esto implica que la rutina de salida del programa, debe incluir instrucciones para deshabilitar el bit de interrupción del reloj en el registro de control correspondiente o resetear el contador a cero. De no hacerse, la interrupción ocurriría innecesariamente mientras el procesador está inactivo o ejecutando otra tarea diferente, provocando un salto a una rutina de servicio que intentaría gestionar un proceso que ya no existe en la tabla de ejecución, lo que desperdiciaría ciclos de reloj y podría causar errores de consistencia en el estado del sistema.

K. Análisis y Discusión de los Resultados

Durante la realización de los ejercicios, brilla el análisis acerca del como se comportan los dispositivos de entrada y de salida, entendiendo el porque y el como de los diferentes enfoques que hay para comunicarnos con ellos, si bien en ambos se utilizo la misma tecnica, sondeo, hay diferencias puntuales. Por un lado, en el ejercicio del semaforo la interrupción viene dada por la introducción de una de dos teclas, el ESC (representando por el numero 27 debido a su posición en la tabla ASCII), y la tecla s (representada por el numero 115 tambien por su posición en la tabla ASCII), en este caso, es ideal el uso de la tecnica de *polling* pues solo tenemos que estar verificando la información del dispositivo de entrada (que en este caso es el teclado), hasta que esta sea uno de los casos validos, es decir, una de las teclas de interes. En contraste, en el ejercicio del *buffer*, se presenta como uno ideal para la interrupciones generadas por hardware, ejecutando asincronamente, un contador que cada 20s, interrumpe el programa para mostrar el *buffer* circular de caracteres almacenados durante ese intervalo de tiempo, y si bien, al final no se utilizaron interrupciones reales pues el fin del ejercicio era simularlas, el mismo hacer ver la utilidad de las mismas en entornos de ejecución reales afuera del simulador.